

Ames Housing Price Predictor

Data Sources, Feature Explanations & Methodology

This document explains — for each tab and section of the deployed Streamlit app — **exactly where every piece of data comes from**, how features are defined, and why each modelling choice was made. Screenshots of each app tab should be inserted in the placeholder boxes below.

■ App URL: <https://ames-housing-by-nezky.streamlit.app> | Author: Neža Podpeskar | Dataset: Ames Housing (Dean De Cock, 2011)

TAB 1 — Predictor

[PASTE SCREENSHOT OF TAB 1 — PREDICTOR (INPUT FORM + RESULTS) HERE — take a screenshot of your app in the browser, crop to just that tab, and insert it here]

What the user sees: A form with ~8 input fields, a Predict button, and two result cards (predicted SalePrice and Premium Home Yes/No).

Where does each input field's data come from?

Input Field	Source of field name / values	How it enters the model
Overall Quality (1–10)	Original Ames Housing dataset column 'Overall Qual'. Values 1–10 assigned by the City of Ames Assessor's Office during property assessment.	Passed directly as a numeric value to the Pipeline's ColumnTransformer → StandardScaler.
Gr Liv Area (sq ft)	Original dataset column 'Gr Liv Area' — above-ground living area in square feet, measured by the assessor.	Numeric input → StandardScaler. Also used to compute the engineered feature TotalSF.
Total Bsmt SF	Original dataset column 'Total Bsmt SF' — total basement area in square feet.	Numeric input → StandardScaler. Also contributes to the engineered TotalSF.
Garage Cars	Original dataset column 'Garage Cars' — number of cars the garage can fit, from assessor records.	Numeric input → StandardScaler.
Year Built	Original dataset column 'Year Built' — year of original construction, from county records.	Numeric input → StandardScaler. Also used to compute engineered HouseAge = Yr Sold – Year Built.
Neighborhood	Original dataset column 'Neighborhood' — physical location within Ames city limits, using official City of Ames Assessor's geographic codes (e.g. NAmes, CollgCr, StoneBr). See full explanation below.	Text → OneHotEncoder (fit on training data) → 25 binary 0/1 columns, one per neighborhood.
Full Bath	Original dataset column 'Full Bath' — count of full bathrooms above grade, from assessor records.	Numeric input → StandardScaler.
Bedroom AbvGr	Original dataset column 'Bedroom AbvGr' — bedrooms above grade (not basement), from assessor records.	Numeric input → StandardScaler.

What about the other ~82 columns the model needs?

The model's Pipeline was trained on 90 feature columns total. The user only enters 8 — the remaining ~82 are filled automatically from **defaults.pkl**, a file saved during training that contains the median value (for numeric columns) or most common category (for text columns) computed from the 2,341 training houses. This means the app always passes a complete, valid 90-column row to the model, no matter which fields the user left at their default.

■ defaults.pkl data source: training split only (2,341 of 2,927 rows). The 586 test-set houses were never used to compute these defaults, preserving leakage-safety.

Where do the RESULTS come from?

- **Predicted SalePrice** — the reg_model.pkl Pipeline (ColumnTransformer preprocessing + tuned XGBoost Regressor) applies its learned weights to the user's input row and returns a continuous dollar prediction. No lookup from past sales — purely mathematical application of patterns learned from training data.
- **Premium Home Yes/No** — the clf_model.pkl Pipeline returns a probability (0–1) that the home is in the top 25% by price. Yes if probability > 0.5. The threshold definition (\$213,500 = 75th percentile) was computed from the training split only.
- **Confidence %** — clf_model.predict_proba() returns the raw probability; displayed as a percentage alongside the Yes/No result.
- **Ideal Buyer Match** — simple if/else logic on the predicted price range, hardcoded in app.py. Not from the model — just descriptive text triggered by which price band the prediction falls into.

NEIGHBOURHOOD DATA — DETAILED EXPLANATION

[PASTE SCREENSHOT OF NEIGHBOURHOOD DROPDOWN (PREDICTOR TAB) HERE — take a screenshot of your app in the browser, crop to just that tab, and insert it here]

This is the question most people ask: the model seems to 'know' about neighbourhoods — but where does that knowledge actually come from? The answer involves three distinct layers, each with a different source:

- **Layer 1 — the neighbourhood NAMES (where they come from):** The 25 neighbourhood codes (NAmes, CollgCr, StoneBr, etc.) come directly from the **original Ames Housing dataset**, which in turn uses the official geographic neighbourhood designations of the **City of Ames Assessor's Office** in Iowa, USA. These are real administrative boundaries — the same codes appear on property tax records and city planning maps. Dean De Cock (the dataset's creator) simply included the assessor's own neighbourhood label for each sale.
- **Layer 2 — the neighbourhood ENCODING (how names become numbers):** Machine learning models cannot use text like 'NAmes' directly — they only understand numbers. So the Pipeline's OneHotEncoder (fit during training) creates one binary column per neighbourhood: e.g. 'Neighborhood_NAmes' = 1 if this is NAmes, 0 otherwise, and 24 other columns are all 0. This encoding was **fit only on the training data** — meaning the encoder learned which 25 neighbourhood categories exist from the 2,341 training houses, not from the full dataset.
- **Layer 3 — the neighbourhood PRICE EFFECT (what the model learned):** During XGBoost training, the model learned how much each neighbourhood binary column pushes the predicted price up or down. For example, StoneBr (Stone Brook) homes commanded a large positive weight — meaning homes in StoneBr tend to sell for significantly more than average, all else equal. This effect was learned purely from past sales patterns in the 2,341 training houses. No external property database, no Zillow, no real estate API — only the historical Ames sales records.

■ Key point: when a user selects 'StoneBr' in the app, the app doesn't look up StoneBr property prices anywhere. It passes a binary 1 in the StoneBr column to the already-trained XGBoost model, which applies the weight it learned during training. The 'knowledge' of StoneBr being expensive is frozen inside the model weights, not fetched live from any database.

REGRESSION FEATURES — Predicting SalePrice

[PASTE SCREENSHOT OF TAB 1 — PREDICTOR RESULT SHOWING PREDICTED SALEPRICE HERE — take a screenshot of your app in the browser, crop to just that tab, and insert it here]

The regression model (XGBoost Regressor, tuned via 5-fold cross-validated GridSearchCV) was trained to predict **SalePrice** — the actual recorded sale price of a home in Ames, Iowa. It uses all 90 feature columns listed below, grouped by type:

Raw features from the original dataset (selected subset):

Feature	What it measures	Data source
Overall Qual	1–10 build/finish quality rating	City of Ames Assessor
Gr Liv Area	Above-ground living area (sq ft)	City of Ames Assessor
Total Bsmt SF	Total basement area (sq ft)	City of Ames Assessor
Garage Cars	Garage capacity (# cars)	City of Ames Assessor
Garage Area	Garage area (sq ft)	City of Ames Assessor
Year Built	Original construction year	County records
Year Remod/Add	Remodel/addition year (= Year Built if never remodelled)	County records
Full Bath	Full bathrooms above grade	City of Ames Assessor
1st Flr SF	First floor area (sq ft)	City of Ames Assessor
Lot Area	Lot size (sq ft)	City of Ames Assessor
Neighborhood	Location within Ames (25 codes)	City of Ames Assessor
Exter Qual	Exterior material quality (Ex/Gd/TA/Fa/Po)	City of Ames Assessor
Kitchen Qual	Kitchen quality (Ex/Gd/TA/Fa/Po)	City of Ames Assessor
Bsmt Qual	Basement height/quality rating	City of Ames Assessor
Fireplace Qu	Fireplace quality (None if no fireplace)	City of Ames Assessor
... + ~75 more columns	All remaining structural, condition, and sale-type fields	City of Ames Assessor / county records

Engineered features (calculated from raw data, not in original dataset):

Engineered Feature	Formula	Why created
TotalSF	Total Bsmt SF + 1st Flr SF + 2nd Flr SF	Combined total living space — a single, stronger size signal than 3 separate columns
HouseAge	Yr Sold – Year Built	Age at time of sale — older homes tend to sell for less; raw year is less intuitive than age
RemodAge	Yr Sold – Year Remod/Add (clipped at 0)	Years since last remodel — recently renovated homes command premiums

Engineered Feature	Formula	Why created
TotalBath	Full Bath + 0.5×Half Bath + Bsmt Full Bath + 0.5×Bsmt Half Bath	All bathrooms combined, weighted by type — standard real-estate convention
Qual_x_SF	Overall Qual × TotalSF	Interaction term: a big AND high-quality home is worth more than either factor alone
AgeCategory	pd.cut(HouseAge, bins=[-1,10,30,60,200])	Binned age group: New / Recent / Established / Old — captures non-linear age effects
TotalPorchSF	Sum of all porch/deck columns	Combined outdoor space — scattered across 5 small columns in original data
HasPool / HasGarage / HasFireplace / Has2ndFloor	1 if area > 0 (or count > 0), else 0	Binary flags — sometimes presence/absence matters more than exact size

■ All engineered features are computed from existing columns by arithmetic — no external data needed. They were created before the train/test split (pure arithmetic does not cause data leakage). The Pipeline then scales/encodes them the same way as raw features.

Feature selection — Lasso:

- Applied Lasso regression (alpha=100) inside the same leakage-safe Pipeline — fit only on X_train (2,341 rows).
- Result: 106 of 343 post-encoding features retained (rest shrunk to exactly 0).
- Top survivors by influence: **Qual_x_SF** (interaction term, strongest), **Neighborhood_StoneBr / Crawford / NoRidge** (location premiums), **Kitchen Qual_Ex / Exter Qual_Ex / Bsmt Qual_Ex** (top-quality grades), **HouseAge** (negative: older = cheaper).
- Source of selection: training data only — the Lasso never saw the 586 test-set houses.

CLASSIFICATION FEATURES — Predicting Premium Home

[PASTE SCREENSHOT OF TAB 1 — PREDICTOR RESULT SHOWING PREMIUM HOME YES/NO HERE — take a screenshot of your app in the browser, crop to just that tab, and insert it here]

The classification model (XGBoost Classifier, same preprocessing Pipeline) predicts **premium_home** — a binary target we derived from SalePrice. It uses the same 90-column feature set as regression, so all feature sources above apply equally here. What differs is the target definition:

How premium_home was derived:

- **Definition:** premium_home = 1 if SalePrice > 75th percentile of training prices, else 0.
- **Threshold:** \$213,500 — computed as train_df['SalePrice'].quantile(0.75) on the 2,341 training rows ONLY. This exact number was then applied to both train and test sets.
- **Why training data only:** Computing the threshold on the full 2,927-row dataset would have used test-set house prices to define the test-set labels — a form of data leakage. Using only the training 75th percentile means the test-set labels were defined without any knowledge of the test houses.
- **Class balance:** 75.0% non-premium / 25.0% premium in train; 75.1% / 24.9% in test — confirming the stratified split worked correctly.

- **Why this question:** A binary 'is this premium?' label directly maps to a real decision buyers and agents face (is this home competing in the luxury/premium segment?), and it can be derived cleanly from the same dataset without external data sources.

■ `premium_home` does NOT exist in the raw Ames Housing dataset. It was created specifically for this project, derived from `SalePrice` using a training-data-only threshold. This is exactly the 'derive target from data' pattern described in the course assignment brief.

TAB 2 — Insights Dashboard

[PASTE SCREENSHOT OF TAB 2 — INSIGHTS DASHBOARD (LEADERBOARD + FEATURE IMPORTANCE CHART) HERE — take a screenshot of your app in the browser, crop to just that tab, and insert it here]

Where does each element in this tab come from?

- **Regression leaderboard table** — hardcoded in `app.py` from the actual test-set evaluation run in the analysis notebook (`01_eda_and_preprocessing.ipynb`). Numbers come from comparing model predictions against 586 truly unseen test houses. R^2 , MAE, RMSE all computed on `y_test_reg` (actual `SalePrice`) vs each model's predictions.
- **Classification leaderboard table** — same source: test-set evaluation on 586 houses. Accuracy, Precision, Recall, F1 from `sklearn.metrics`; ROC-AUC from `predict_proba()` probabilities vs actual `premium_home` labels.
- **Dummy baseline rows** — `DummyRegressor(strategy='mean')` predicts the training average for every house; `DummyClassifier(strategy='most_frequent')` always predicts 0 (not premium). Both fit on `X_train`, evaluated on `X_test`. Kept in the table to show the floor — any real model must beat these.
- **Feature importance bar chart** — comes from `reg_model.named_steps['model'].feature_importances_` at runtime when the app loads. This is the XGBoost model's own internal measure of how much each feature reduced prediction error across all 300 trees. Feature names come from the Pipeline preprocessor's `get_feature_names_out()`. Data source: learned from training data, stored in the pickle file.
- **Written takeaways / honest notes** — hardcoded text in `app.py`, written by the author based on analysis findings. Not generated dynamically.

TAB 3 — About & Methodology

[PASTE SCREENSHOT OF TAB 3 — ABOUT & METHODOLOGY TAB HERE — take a screenshot of your app in the browser, crop to just that tab, and insert it here]

- **Dataset description text** — written by the author; facts sourced from Dean De Cock's original 2011 paper 'Ames, Iowa: Alternative to the Boston Housing Data Set.'
- **Regression / Classification explanations** — authored explanations, no live data.
- **Feature table** — hardcoded in `app.py` from the analysis notebook's Lasso results and domain knowledge about each column.
- **XGBoost explanation** — authored text. Performance numbers ($R^2=0.944$, etc.) from test-set evaluation in the notebook.

- **Ideal Buyer Match widget** — purely app logic (if/else on predicted price range). The price ranges themselves are hardcoded thresholds based on the dataset's known quartile structure (\$130k / \$213.5k / \$350k), not from a live source.
- **'Where was the model tested?' section** — factual description of the train/test split: 2,341 training rows / 586 test rows, stratified by SalePrice quartile. All reported accuracy numbers come from the 586 unseen test rows only.

SIDEBAR

[PASTE SCREENSHOT OF SIDEBAR (LOGO + ABOUT SECTION + GITHUB LINK) HERE — take a screenshot of your app in the browser, crop to just that tab, and insert it here]

- **Logo** — assets/logo.png, a local image file included in the project repository. No external source — created/provided by the author.
- **'About this project' blurb** — hardcoded text in app.py.
- **GitHub link** — placeholder link in app.py pointing to the project's public GitHub repository (to be updated with the real URL before submission).
- **App framework** — Streamlit, deployed on Streamlit Community Cloud (share.streamlit.io). Free tier, Python-based, model runs in-process.

SUMMARY — Where Every Piece of Data Comes From

App element	Data source	When created
Input field values (user-entered)	User types/selects them in the browser	At prediction time
Background column defaults (~82 fields)	defaults.pkl — median/mode of training split	Saved after model training
Neighbourhood names (dropdown list)	Original Ames Housing dataset (City of Ames Assessor codes)	In dataset since 2006–2010
Neighbourhood price effect	Learned by XGBoost from training data patterns	During model training
Predicted SalePrice	reg_model.pkl Pipeline applied to user input	At prediction time (live)
Premium Home Yes/No + confidence	clf_model.pkl Pipeline applied to user input	At prediction time (live)
Ideal Buyer Match description	Hardcoded if/else logic in app.py on predicted price	At prediction time
Regression leaderboard numbers	Test-set evaluation in analysis notebook (586 unseen houses)	During model evaluation
Classification leaderboard numbers	Test-set evaluation in analysis notebook (586 unseen houses)	During model evaluation
Feature importance chart	XGBoost model's feature_importances_ from reg_model.pkl	Computed at app startup from pickle

App element	Data source	When created
Engineered features (TotalSF etc.)	Arithmetic from original dataset columns	Pre-split feature engineering step
Preprocessing (scaling, encoding)	ColumnTransformer fit on training data only	During model training
Logo	assets/logo.png — local file, provided by author	Static asset in repo
All explanatory text	Authored by Neža Podpeskar based on analysis findings	Written for this project

■ **IMPORTANT** — no data is fetched live from external APIs, databases, or websites when the app runs. Every number, prediction, and chart either comes from the pre-trained pickle files (reg_model.pkl, clf_model.pkl, defaults.pkl), hardcoded values from the analysis notebook, or is computed on the fly from user inputs + the loaded models. The app is fully self-contained.